

Dedicated Claude Code Machine

Setup guide — Dell OptiPlex 3050 Micro · Ubuntu Server 26.04 LTS

Hardware	Dell OptiPlex 3050 Micro — Intel Core i5-7500T, 16 GB RAM, 512 GB NVMe SSD
Operating system	Ubuntu Server 26.04.1 LTS (headless, supported to April 2031)
Access	SSH from your laptop; no monitor or keyboard after installation
Boot keys	F2 = BIOS setup · F12 = boot menu

Claude Code's work happens on Anthropic's servers, so this machine only needs to run a terminal, git, your toolchains, and your test suite. The published requirement is 4 GB RAM and an x64 processor, which this comfortably exceeds. Ubuntu Server rather than Desktop keeps idle memory use around 1 GB instead of 4 GB or more.

Micro chassis notes. This model has no PCIe slot, so a graphics card is not an option — that's fine, since nothing in this guide needs one. Memory is SODIMM across two slots, maximum 32 GB. Idle draw is roughly 15 W, which makes leaving it running permanently effectively free.

Phase 0 — BIOS and hardware preparation

Do this before touching the installer. Power on and press `F2`.

1. **Set SATA Operation to AHCI.** These machines very often ship set to `RAID On`, and the Ubuntu installer will then show no disks whatsoever. This is by far the most common way this install goes wrong.
2. **Power Management → AC Recovery → Power On.** The machine then comes back by itself after a power cut, which matters for a box you rely on being reachable.
3. **Disable Deep Sleep Control.** Deep Sleep prevents Wake-on-LAN and remote power-on from working.
4. **Enable Wake-on-LAN** if you want to be able to wake it remotely. Optional.
5. **Leave Secure Boot enabled.** Ubuntu handles it without any special steps.
6. **Update the BIOS** while you still have a screen attached. Download the latest `.exe` from Dell's OptiPlex 3050 support page, copy it to a FAT32 USB stick, then press `F12` at boot and choose *BIOS Flash Update*. No operating system required.

Phase 1 — Install Ubuntu Server

1. Download **Ubuntu Server 26.04.1 LTS** — the server ISO, not Desktop — from `ubuntu.com/download/server`.
2. Write it to a USB stick. Rufus on Windows, or on Linux:

```
sudo dd if=ubuntu-26.04.1-live-server-amd64.iso of=/dev/sdX bs=4M status=progress conv=fsync
```

3. Boot the machine, press `F12`, select the USB stick.
4. Work through the installer:
 - **Network:** accept DHCP, then afterwards set a *DHCP reservation on your router* so the address never changes. Simpler and more robust than configuring a static address on the host.
 - **Storage:** choose "Use an entire disk" and tick **Set up this disk as an LVM group**. LVM lets you snapshot before risky operations later.
 - **Profile:** pick a short hostname you won't mind typing often, such as `agent`.
 - **Tick "Install OpenSSH server."** Do not skip this — everything from here on is headless.
 - **Import SSH identity → from GitHub**, entering your GitHub username. This pulls your public keys automatically.
 - **Featured snaps:** skip all of them.
5. Reboot, remove the USB stick, disconnect the monitor. Connect from your laptop:

```
ssh yourname@agent.local    # or the IP address
```

Phase 2 — Base system

```
# Updates
sudo apt update && sudo apt full-upgrade -y

# Automatic security updates
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades

# Timezone
sudo timedatectl set-timezone Europe/Vienna

# Firewall
sudo ufw allow OpenSSH
sudo ufw enable
```

Then restrict SSH to key authentication. Edit `/etc/ssh/sshd_config`:

```
PasswordAuthentication no
PermitRootLogin no
```

```
sudo systemctl restart ssh
```

Before closing this terminal, open a second one and confirm you can still log in. If key authentication isn't working, the first session is your only way back in.

Phase 3 — Development toolchain

```
sudo apt install -y build-essential git curl wget unzip \
    ripgrep fd-find jq tmux htop ncdu

# GitHub CLI
sudo apt install -y gh
gh auth login      # also offers to configure git credentials
```

Git identity:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase true
```

Node.js

Use a version manager rather than apt, so individual projects can pin their own versions.

```
curl -fsSL https://fnm.vercel.app/install | bash
exec $SHELL
fnm install --lts
fnm default lts-latest
```

Python

`uv` handles interpreters and virtual environments in a single tool.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Docker (optional)

The cleanest way to contain a risky agent run to a single directory.

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker
```

Add anything else your projects need — Go, Rust via rustup, a database, and so on.

Phase 4 — Install Claude Code

Two options. Pick one.

Option A — Native installer (auto-updates in the background)

```
curl -fsSL https://claude.ai/install.sh | bash
```

Option B — apt repository (updates arrive through `apt upgrade`)

```
sudo apt install -y curl gnupg
sudo install -d -m 0755 /etc/apt/keyrings
sudo curl -fsSL https://downloads.claude.ai/keys/claude-code.asc \
  -o /etc/apt/keyrings/claude-code.asc

# Verify the key before trusting it. Expected fingerprint:
# 31DDDE24DDFAB679F42D7BD2BAA929FF1A7ECACE
gpg --show-keys /etc/apt/keyrings/claude-code.asc

echo "deb [signed-by=/etc/apt/keyrings/claude-code.asc] https://downloads.claude.ai/claude-code/apt/stable stable main" \
  | sudo tee /etc/apt/sources.list.d/claude-code.list
sudo apt update && sudo apt install claude-code
```

For an unattended machine, Option B has the advantage that the version won't change under you between sessions. Verify either way:

```
claude --version      # prints e.g. 2.1.211 (Claude Code)
claude doctor         # read-only diagnostics: install health, settings errors
```

If you get `claude: command not found`, add this to `~/.bashrc`:

```
export PATH="$HOME/.local/bin:$PATH"
```

Authenticate

Claude Code requires a Pro, Max, Team, Enterprise, or Console account. The free Claude.ai plan does not include access.

```
cd ~/some-project
claude
```

On a headless machine it prints a login URL. Open that in the browser on your laptop, complete the flow, and paste the code back. The session token is stored in `~/.claude.json`.

Phase 5 — Remote workflow

The point of a dedicated machine is that a long run keeps going after your laptop sleeps. That requires a terminal multiplexer.

```
sudo apt install -y tmux
```

A minimal `~/.tmux.conf`:

```
set -g mouse on
set -g history-limit 50000
set -g default-terminal "tmux-256color"
```

```
tmux new -s wow      # one named session per project
# Ctrl-b then d      -> detach; work continues running
tmux ls              # list sessions
tmux attach -t wow   # reattach later, from anywhere
```

Access from outside your network

Tailscale avoids opening any port on your router.

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --ssh
```

Editor integration

Install the *Remote - SSH* extension in VS Code on your laptop, connect to the machine, and run `claude` in the integrated terminal. Files are edited directly on the server, with no syncing or mounted network drives.

Unreliable connections

```
sudo apt install -y mosh # then connect with mosh instead of ssh
```

Phase 6 — Guardrails

A machine dedicated to running an agent should be one you'd be relaxed about losing.

1. **Keep secrets off it.** No password manager exports, no unrelated SSH keys, no production `.env` files. Where a repository needs credentials, scope them to that repository.
2. **Everything in git, pushed often.** This is your best undo button. Use `git worktree` to give parallel sessions separate directories on the same repository.
3. **Configure updates deliberately** in `~/.claude/settings.json`:

```
{
  "autoUpdatesChannel": "stable"
}
```

The stable channel trails by about a week and skips releases with known major regressions — a sensible trade for a machine you don't watch closely.

4. **Run genuinely risky work in a container.** A devcontainer, or plain `docker run -v $PWD:/work`, limits the blast radius to one directory.
5. **Back up.** At minimum a nightly job pushing to a NAS or object storage:

```
sudo apt install -y restic
```

With LVM from Phase 1 you can also snapshot before a large refactor and roll back if it goes badly.

Phase 7 — First project

```
mkdir -p ~/code && cd ~/code
gh repo clone Ruban-Creator/wow-gearing-tool
cd wow-gearing-tool
tmux new -s wow
claude
```

Inside the session, run `/init`. Claude Code writes a `CLAUDE.md` describing the project's structure, commands, and conventions. That file is the highest-leverage thing on the machine: it's what stops every future session from rediscovering the codebase from scratch. Commit it.

Quick reference

Task	Command
BIOS setup / boot menu	<code>F2 / F12</code>
Check installation health	<code>claude doctor</code>
Update Claude Code manually	<code>claude update</code>
Start a named session	<code>tmux new -s name</code>
Detach from a session	<code>Ctrl-b then d</code>
Reattach to a session	<code>tmux attach -t name</code>
List sessions	<code>tmux ls</code>
System update	<code>sudo apt update && sudo apt full-upgrade</code>
Confirm chassis model	<code>sudo dmidecode -s system-product-name</code>

Claude Code documentation: <https://code.claude.com/docs/en/setup>

Later additions worth considering: 32 GB of SODIMM memory if 16 GB starts limiting builds and containers, and a second machine with a discrete GPU if you decide to run local models — this chassis has no PCIe slot, so that cannot happen here.